

# Tugas Model Warna

Penulis: Kenny Gabriel Manurung  
Dosen: Arta Kusuma Hernanda  
NRP : 5024241022  
Nama Kelas: Pengolahan Citra dan Video (A)

## Kode yang Digunakan

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# TUGAS 4 – MODEL WARNA
# KONVERSI RGB -> CMYK, HSI, DAN HSV

# KONVERSI RGB -> CMYK

def rgb_to_cmyk(rgb):

    # Normalisasi RGB dari 0-255 menjadi 0-1
    rgb_normalized = rgb.astype(np.float32) / 255.0

    # Pisahkan channel RGB
    r = rgb_normalized[:, :, 0]
    g = rgb_normalized[:, :, 1]
    b = rgb_normalized[:, :, 2]

    # RGB -> CMY

    c = 1 - r
    m = 1 - g
    y = 1 - b

    # Menentukan nilai K

    # K adalah nilai minimum dari C, M, dan Y
    k = np.minimum(
        np.minimum(c, m),
        y
    )
```

```
# Menghitung C, M, Y setelah K diperoleh
```

```
denominator = 1 - k
```

```
# Membuat array kosong
```

```
cmyk_c = np.zeros_like(c)
```

```
cmyk_m = np.zeros_like(m)
```

```
cmyk_y = np.zeros_like(y)
```

```
# Menghindari pembagian dengan nol
```

```
mask = denominator != 0
```

```
cmyk_c[mask] = (  
    (c[mask] - k[mask])  
    / denominator[mask]  
)
```

```
cmyk_m[mask] = (  
    (m[mask] - k[mask])  
    / denominator[mask]  
)
```

```
cmyk_y[mask] = (  
    (y[mask] - k[mask])  
    / denominator[mask]  
)
```

```
return cmyk_c, cmyk_m, cmyk_y, k
```

```
# KONVERSI RGB -> HSI
```

```
def rgb_to_hsi(rgb):
```

```
# Konversi ke float
```

```
rgb = rgb.astype(np.float32)
```

```
# Pisahkan channel RGB
```

```
r = rgb[:, :, 0]
```

```
g = rgb[:, :, 1]
```

```
b = rgb[:, :, 2]
```

```
# 1. INTENSITY
```

```
#  $I = (R + G + B) / 3$ 
```

```
intensity = (  
    r + g + b  
) / 3.0
```

```
# 2. SATURATION
```

```
# Nilai minimum RGB
```

```

minimum = np.minimum(
    np.minimum(r, g),
    b
)

# Jumlah RGB
total = r + g + b

# Membuat array saturation
saturation = np.zeros_like(intensity)

# Menghindari pembagian dengan nol
mask = total != 0

# Rumus:

#  $S = 1 - (3 * \min(R, G, B)) / (R+G+B)$ 

saturation[mask] = (
    1
    - (
        3 * minimum[mask]
        / total[mask]
    )
)

# 3. HUE

# Pembilang
numerator = 0.5 * (
    (r - g) +
    (r - b)
)

# Penyebut
denominator = np.sqrt(
    (r - g) ** 2 +
    (r - b) * (g - b)
)

# Menghindari pembagian dengan nol
denominator = np.maximum(
    denominator,
    1e-10
)

# Nilai  $\cos(\theta)$ 
cosine = numerator / denominator

# Membatasi nilai antara -1 sampai 1
cosine = np.clip(
    cosine,
    -1.0,
    1.0
)

#  $\cos^{-1}$  menghasilkan radian
theta = np.arccos(cosine)

```

```

# Radian -> derajat
theta = np.degrees(theta)

# Menentukan Hue

# Jika G >= B:
# H = theta

# Jika G < B:
# H = 360 - theta

hue = np.where(
    g >= b,
    theta,
    360 - theta
)

# Untuk warna grayscale:
# R = G = B

# Hue dianggap 0
gray_mask = (
    (r == g) &
    (g == b)
)

hue[gray_mask] = 0

return hue, saturation, intensity

```

*# KONVERSI RGB -> HSV*

```

def rgb_to_hsv_manual(rgb):

    # Normalisasi RGB dari 0-255 menjadi 0-1
    rgb_normalized = (
        rgb.astype(np.float32)
        / 255.0
    )

    # Pisahkan channel RGB
    r = rgb_normalized[:, :, 0]
    g = rgb_normalized[:, :, 1]
    b = rgb_normalized[:, :, 2]

    # Nilai maksimum dan minimum

    maximum = np.maximum(
        np.maximum(r, g),
        b
    )

    minimum = np.minimum(
        np.minimum(r, g),

```

```

        b
    )

# Selisih maksimum dan minimum
delta = maximum - minimum

# 1. VALUE

#  $V = \max(R, G, B)$ 
value = maximum

# 2. SATURATION

saturation = np.zeros_like(value)

# Jika maximum tidak nol
mask = maximum != 0

#  $S = \text{delta} / \text{maximum}$ 
saturation[mask] = (
    delta[mask]
    / maximum[mask]
)

# 3. HUE

hue = np.zeros_like(value)

# Delta = 0 berarti grayscale
non_zero = delta != 0

# Jika R adalah nilai maksimum

mask_r = (
    non_zero &
    (maximum == r)
)

hue[mask_r] = (
    60 *
    (
        (g[mask_r] - b[mask_r])
        / delta[mask_r]
    )
)

# Jika G adalah nilai maksimum

mask_g = (
    non_zero &
    (maximum == g)
)

```

```
)
hue[mask_g] = (
    60 *
    (
        (
            (b[mask_g] - r[mask_g])
            / delta[mask_g]
        ) + 2
    )
)
```

*# Jika B adalah nilai maksimum*

```
mask_b = (
    non_zero &
    (maximum == b)
)
```

```
hue[mask_b] = (
    60 *
    (
        (
            (r[mask_b] - g[mask_b])
            / delta[mask_b]
        ) + 4
    )
)
```

*# Jika Hue negatif, tambahkan 360*  
hue[hue < 0] += 360

```
return hue, saturation, value
```

*# PROGRAM UTAMA*

*# Nama file gambar*  
image\_path = "rb.png"

*# MEMBACA GAMBAR*

```
image_bgr = cv2.imread(image_path)
```

*# Memeriksa apakah gambar berhasil dibaca*  
**if** image\_bgr **is** None:

```
    print ("ERROR: _Gambar_tidak_ditemukan!")
    print (
        f"Pastikan_file_{image_path}"
        "berada_di_folder_yang_sama_dengan_program."
    )
```

```

    exit ()

# BGR -> RGB

# OpenCV membaca gambar sebagai BGR.
# Karena rumus menggunakan RGB,
# maka gambar dikonversi terlebih dahulu.

image_rgb = cv2.cvtColor(
    image_bgr,
    cv2.COLOR_BGR2RGB
)

# KONVERSI MODEL WARNA

# RGB -> CMYK
c, m, y, k = rgb_to_cmyk(
    image_rgb
)

# RGB -> HSI
h_hsi, s_hsi, i_hsi = rgb_to_hsi(
    image_rgb
)

# RGB -> HSV
h_hsv, s_hsv, v_hsv = rgb_to_hsv_manual(
    image_rgb
)

# INFORMASI GAMBAR

height, width, channels = image_rgb.shape

print ("=" * 60)
print ("TUGAS_4_-_MODEL_WARNA")
print ("=" * 60)

print (f"Ukuran_gambar: {width}x{height}")
print (f"Jumlah_channel_RGB: {channels}")

print ()
print ("Model_warna_yang_digunakan:")
print ("_RGB")
print ("_CMYK")
print ("_HSI")
print ("_HSV")

# NILAI PIXEL TENGAH

```

```

center_y = height // 2
center_x = width // 2

rgb_pixel = image_rgb[
    center_y,
    center_x
]

r, g, b = rgb_pixel

print()
print("=" * 60)
print("CONTOH_KONVERSI_PIXEL_TENGAH")
print("=" * 60)

print(
    f"Lokasi_pixel: ({center_x}, {center_y})"
)

# RGB

print()
print("RGB")

print(f"R={r}")
print(f"G={g}")
print(f"B={b}")

# CMYK

print()
print("CMYK")

print(
    f"C={c[center_y, center_x]*100:.2f}%"
)

print(
    f"M={m[center_y, center_x]*100:.2f}%"
)

print(
    f"Y={y[center_y, center_x]*100:.2f}%"
)

print(
    f"K={k[center_y, center_x]*100:.2f}%"
)

# HSI

```

```

print()
print("HSI")

print(
    f"H={h_hsi[center_y, center_x]:.2f}°"
)

print(
    f"S={s_hsi[center_y, center_x]*100:.2f}%"
)

print(
    f"I={i_hsi[center_y, center_x]:.2f}"
)

# HSV

print()
print("HSV")

print(
    f"H={h_hsv[center_y, center_x]:.2f}°"
)

print(
    f"S={s_hsv[center_y, center_x]*100:.2f}%"
)

print(
    f"V={v_hsv[center_y, center_x]*100:.2f}%"
)

print("=" * 60)

# VISUALISASI

# Setiap hasil ditampilkan satu per satu.

# Tutup jendela yang sedang tampil untuk
# melanjutkan ke gambar berikutnya.

# Tidak menggunakan tombol atau sistem slide.

# 1. RGB

plt.figure(figsize=(12, 7))

plt.imshow(image_rgb)

plt.title(

```

```

        "RGB",
        fontsize=18
    )

plt.axis("off")

plt.tight_layout()

plt.show()

```

*# 2. CMYK – CYAN*

```

plt.figure(figsize=(12, 7))

plt.imshow(
    c,
    cmap="gray"
)

plt.title(
    "CMYK_□_Cyan_□(C)",
    fontsize=18
)

plt.axis("off")

plt.tight_layout()

plt.show()

```

*# 3. CMYK – MAGENTA*

```

plt.figure(figsize=(12, 7))

plt.imshow(
    m,
    cmap="gray"
)

plt.title(
    "CMYK_□_Magenta_□(M)",
    fontsize=18
)

plt.axis("off")

plt.tight_layout()

plt.show()

```

*# 4. CMYK – YELLOW*

```
plt.figure(figsize=(12, 7))

plt.imshow(
    y,
    cmap="gray"
)

plt.title(
    "CMYK_ Yellow(Y)",
    fontsize=18
)

plt.axis("off")

plt.tight_layout()

plt.show()
```

*# 5. CMYK – BLACK*

```
plt.figure(figsize=(12, 7))

plt.imshow(
    k,
    cmap="gray"
)

plt.title(
    "CMYK_ Black(K)",
    fontsize=18
)

plt.axis("off")

plt.tight_layout()

plt.show()
```

*# 6. HSI – HUE*

```
plt.figure(figsize=(12, 7))

plt.imshow(
    h_hsi,
    cmap="hsv"
)

plt.title(
    "HSI_ Hue",
    fontsize=18
)

plt.axis("off")

plt.tight_layout()
```

```
plt.show()
```

```
# 7. HSI – SATURATION
```

```
plt.figure(figsize=(12, 7))
```

```
plt.imshow(  
    s_hsi,  
    cmap="gray"  
)
```

```
plt.title(  
    "HSI – Saturation",  
    fontsize=18  
)
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# 8. HSI – INTENSITY
```

```
plt.figure(figsize=(12, 7))
```

```
plt.imshow(  
    i_hsi,  
    cmap="gray"  
)
```

```
plt.title(  
    "HSI – Intensity",  
    fontsize=18  
)
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# 9. HSV – HUE
```

```
plt.figure(figsize=(12, 7))
```

```
plt.imshow(  
    h_hsv,  
    cmap="hsv"  
)
```

```
plt.title(
    "HSV_Hue",
    fontsize=18
)
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

*# 10. HSV – SATURATION*

```
plt.figure(figsize=(12, 7))
```

```
plt.imshow(
    s_hsv,
    cmap="gray"
)
```

```
plt.title(
    "HSV_Saturation",
    fontsize=18
)
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```

*# 11. HSV – VALUE*

```
plt.figure(figsize=(12, 7))
```

```
plt.imshow(
    v_hsv,
    cmap="gray"
)
```

```
plt.title(
    "HSV_Value",
    fontsize=18
)
```

```
plt.axis("off")
```

```
plt.tight_layout()
```

```
plt.show()
```